

Lab 1 report

卢天姿

24300130073

新闻 + 人工智能

September 29, 2026

Contents

1	运行结果	3
2	19 道谜题的函数思路	4
2.1	P1.signMask	4
2.2	P2.bitXor	4
2.3	P3.negativePart	4
2.4	P4.copyByteWithin	4
2.5	P5.logicalShift	4
2.6	P6.swapNibblePairs	5
2.7	P7.secondLowestZeroBit	5
2.8	P8.oddParity	5
2.9	P9.rotateRightBits	5
2.10	P10.roundEvenPow2	5
2.11	P11.midpointTowardFirst	6
2.12	P12.isBetweenEitherOrder	6
2.13	P13.mul5Sat	6
2.14	P14.classifyAdd3	6
2.15	P15.floatScaleThreeHalves	7
2.16	P16.floatRoundEven	7

2.17	P17.float_i2f	7
2.18	P18.bitCount	7
2.19	P19.bitReverse	7
2.20	总结与收获	8
3	参考资料	8

1 运行结果

因为笔者使用 mac 系统，暂未拥有编译环境，所以检验运行结果从 Github 的 Action 界面进行读取，详见Figure 1。

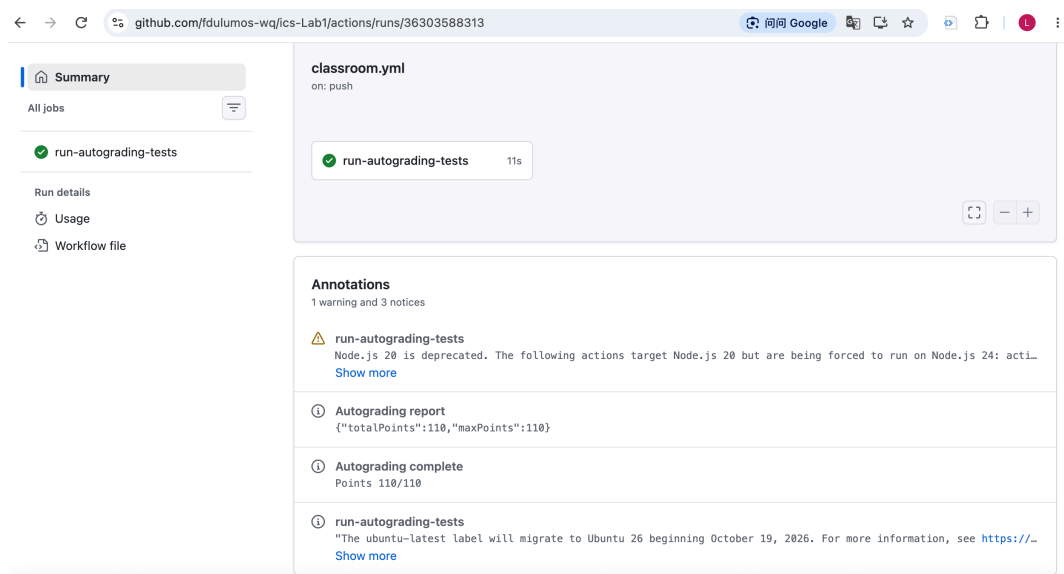


Figure 1: Lab1 的 Action 界面下的 Annotation

自动评分结果：GitHub Actions 正常完成，最终得分为 110/110。

2 19 道谜题的函数思路

具体谜题的碰壁、思考、每一步实现了什么都写在代码中了，以下是谜题的大致思路。

2.1 P1.signMask

1. 本题大意：返回一个 32 位整数只有最高位是 1，其余都是 0（就是十六进制表达下的 0x80000000）。
2. 因为有符号的限制，自然的想到就是将二进制的 1 左移 31 位到最高位即可得到

2.2 P2.bitXor

1. 本题大意：只使用按位取反和按位与，实现按位异或的效果，即相同位返回 0，不同位返回 1。
2. 实现时先用 $x \& y$ 标记两者同为 1 的位置，再用 $(\sim x) \& (\sim y)$ 标记两者同为 0 的位置。把这两类位置取反取交集，剩下的就是 01 和 10，也就是异或结果为 1 的位置。

2.3 P3.negativePart

1. 本题大意：当 x 为负数时返回 $-x$ ，否则返回 0。
2. 将 x 算术右移 31 位得到符号掩码：非负数得到全 0，负数得到全 1。再用 $\sim x + 1$ 计算相反数，并与符号掩码做位与，从而在非负时保留 0，在负数时保留相反数。

2.4 P4.copyByteWithin

1. 本题大意：将 x 的第 src 个字节复制到第 dst 个字节，其余字节不变。
2. 字节编号乘 8 可转化为位移量，所以用左移 3 位得到 $srcShift$ 和 $dstShift$ 。先右移并与 0xFF 提取源字节，再把它左移到目标位置。同时用目标位置上的 0xFF 掩码清空原目标字节，最后用位或拼接结果。

2.5 P5.logicalShift

1. 本题大意：将 x 逻辑右移 n 位，即右移后高位补 0。

2. C 语言中的有符号右移是算术右移，负数高位会补 1。因此先正常右移，再从最高位掩码 $1 \ll 31$ 出发构造“最高 n 位为 0、其余位为 1”的掩码，最后通过位与清除算术右移补出的 1。该构造也能正确处理 $n=0$ 。

2.6 P6.swapNibblePairs

1. 本题大意：交换 x 每个字节中的高四位和低四位。
2. 先由 $0x0F$ 扩展得到 $0x0F0F0F0F$ 。用它提取每个字节的低四位并左移 4 位；高四位则先右移 4 位，再用同一掩码清除跨字节进入的无关位，最后将两部分按位或合并。

2.7 P7.secondLowestZeroBit

1. 本题大意：从最低位向最高位寻找第二个 0，并返回只标记该位置的掩码。
2. 先对 x 取反，使原来的 0 变成 1。利用 $a \& (a-1)$ 清除最低的 1，也就是清除原数最低的 0；因为不能使用减号，所以将 $a-1$ 写成 $a+\sim 0$ 。然后利用 $b \& (-b)$ 隔离剩余的最低 1，即原数第二低的 0。

2.8 P8.oddParity

1. 本题大意：当 x 的 32 位二进制表示中 1 的数量为偶数时返回 1，否则返回 0。
2. 利用异或能够保存奇偶性的特点，将 x 依次与其右移 16、8、4、2、1 位的结果异或，把 32 位的奇偶性逐层折叠到最低位。最低位为 0 表示 1 的数量为偶数，最低位为 1 表示奇数，因此最后对该位取逻辑非。

2.9 P9.rotateRightBits

1. 本题大意：把 x 循环右移 n 位，右侧移出的位需要从左侧重新进入。
2. 先用 $n \& 31$ 得到实际右移量 s ，再用 $(-s) \& 31$ 得到对应左移量。右移部分采用与 P5 相同的方法清除算术右移补出的 1，左移部分保存被挤出的低位，最后将两部分按位或合并。取模后的写法同时处理了 n 为 32 倍数的情况。

2.10 P10.roundEvenPow2

1. 本题大意：将非负整数 x 舍入到最接近的 2^n 倍数；恰好位于中点时，选择除以 2^n 后商为偶数的倍数。

2. 先构造 $a = 2^n$ 、最低 n 位掩码以及中点 $a/2$ 。 $(x >> n) \& 1$ 得到下方倍数之商的奇偶性，偏置量取“中点减 1 再加商的奇偶位”。将偏置加到 x 后清除最低 n 位，即可统一完成普通四舍五入和中点取偶。

2.11 P11.midpointTowardFirst

1. 本题大意：在不溢出的前提下计算 x 与 y 的数学中点；若中点恰好是半整数，则选择更靠近第一个参数 x 的整数。
2. 使用 $(x \& y) + ((x \hat{y}) >> 1)$ 计算向下取整的中点，避免直接计算 $x+y$ 造成溢出。 $(x \hat{y}) \& 1$ 判断是否出现半整数。之后分别处理同号和异号情况，得到 $x > y$ 的全位掩码；仅当中点为半整数且 $x > y$ 时加 1，否则保留向下取整结果。

2.12 P12.isBetweenEitherOrder

1. 本题大意：判断 x 是否位于端点 a 和 b 构成的闭区间内，端点可以按任意顺序给出。
2. 先结合端点符号和 $a-b$ 的符号，构造选择掩码，得到真正的 low 和 $high$ 。随后分别判断 $low \leq x$ 和 $x \leq high$ 。比较时先检查两数是否异号，同号时再使用差值符号，避免减法溢出导致判断错误。两侧条件都成立时返回 1。

2.13 P13.mul5Sat

1. 本题大意：计算 $x*5$ ；如果发生正溢出则返回 INT_MAX ，发生负溢出则返回 INT_MIN 。
2. 先构造临界绝对值 $0x19999999$ ，它是 $INT_MAX/5$ 向下取整的结果。通过符号掩码求出 x 的绝对值并与临界值比较，确定是否溢出。未溢出时用 $(x << 2) + x$ 计算五倍；溢出时根据原符号选择最大值或最小值。

2.14 P14.classifyAdd3

1. 本题大意：判断数学意义上的 $x+y+z$ 是否超出 32 位有符号整数范围；正溢出返回 1，负溢出返回 -1，否则返回 0。
2. 将加法拆成 $x+y$ 和“前一次结果再加 z ”两步。每一步都根据“两个加数同号但结果异号”判断溢出，并把正溢出记为 1、负溢出记为 -1。最后将两次标记相加：方向相同则保留溢出方向，方向相反则互相抵消，表示精确总和重新回到合法范围。

2.15 P15.floatScaleThreeHalves

1. 本题大意：把输入位模式解释为 IEEE 754 单精度浮点数，计算其 $3/2$ 倍，并采用最近偶数舍入；NaN 保持原位模式，正负零保留符号。
2. 先拆出符号位、阶码和尾数。NaN 与无穷直接返回；非规格化数没有隐藏位，因此直接将尾数乘 3 再除 2 并处理舍入。规格化数先补上隐藏的最高位 1，再完成乘 3 和右移。根据被舍弃部分与中点的关系执行最近偶数舍入，并处理尾数进位、重新规格化及结果变为无穷的情况。

2.16 P16.floatRoundEven

1. 本题大意：将浮点数舍入到最近的整数值，并仍以单精度浮点位模式返回；恰好位于两个整数中间时选择偶数，NaN 和无穷保持不变。
2. 先利用阶码区分特殊值、绝对值小于 0.5、介于 0.5 和 1 之间以及已经没有小数位的情况。对于仍含小数位的规格化数，补回隐藏位并计算小数部分占用的位数，用掩码分离保留部分和舍弃部分。当舍弃部分超过一半，或恰好一半且当前整数为奇数时向上舍入，最后处理舍入产生的有效位进位。

2.17 P17.float_i2f

1. 本题大意：将整数 x 转换成单精度浮点数，并返回对应的 IEEE 754 位级表示。
2. 先处理 0，并记录负数的符号和无符号绝对值，从而正确覆盖 INT_MIN。从最高位向下寻找第一个 1，其位置加 127 得到阶码。如果有效数字不超过 24 位，就直接对齐到隐藏位；否则只保留最高 24 位，并根据被舍弃低位执行最近偶数舍入。若舍入产生进位，则右移有效数字并增加阶码。

2.18 P18.bitCount

1. 本题大意：统计 x 的 32 位二进制表示中 1 的数量。
2. 如果允许循环，可以逐位右移并累加最低位，但整数题禁止循环。因此构造 0x55555555、0x33333333、0x0F0F0F0F 等掩码，先同时统计每 2 位中的 1，再依次合并成每 4 位、8 位、16 位和 32 位的计数。层层合并本质上是对逐位累加的并行实现。

2.19 P19.bitReverse

1. 本题大意：将 x 的 32 个二进制位按照顺序完全颠倒。

2. 如果允许循环，可以不断取出最低位并放入持续左移的结果中；本题则使用分组交换。先从低 16 位掩码逐步构造 8 位、4 位、2 位和 1 位分组掩码，再依次交换相邻的 1 位组、2 位组、4 位组、8 位组，最后交换高低 16 位。经过五轮后，全部 32 位的顺序被反转，同时满足 34 个操作符的限制。

2.20 总结与收获

通过这 19 道题不同的 rating，笔者总结出了一些做题时常见的思路与心得：

1. **在涉及运算时一定要考虑符号、溢出这两个重要的边界。** 尤其是涉及顺序、多位运算，正负号会出现抵消、左移右移效果不同这些容易被忽略的情况；而溢出更是需要拎出来讨论的，我们要在构造掩码、或者设置函数的时候提前考虑到这一点，在构造函数时就要有意识的进行验证。
2. **构造掩码很重要。** 很多时候涉及逻辑移动、只改变几位的时候，选择构造合适的掩码是至关重要的。而且要构造出位置比较巧妙的单元，利用左移右移就能实现复制。
3. **要有分块意识。** 很多时候涉及调换顺序、计算整体个数这样的题目，将 32 位拆开分块是非常有效的一种方式。可以拆开之后再利用位或将它拼凑起来，也可以每几位（1、2、4、8 16）一计算最后合并，这样就能替代 c 语言当中的循环。
4. **没有减法。** 有的时候做着做着就想到了 $-x$ ，在实际操作中要换成 $\sim x + 1$ 。判断大小都转化为做差

3 参考资料

- 复旦大学《计算机系统基础》课程，Lab 1: Data Lab 实验文档：<https://ics-26fall-fdu.github.io/labs/lab1-data-lab/>
- 课程实验模板仓库：<https://github.com/ICS-26Fall-FDU/DataLab>
- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd Edition.
- 大模型用于检查边界情况，因为 mac 原生系统不支持 test 函数的实现。